

Received 18 March 2022; revised 25 May 2022; accepted 10 June 2022; date of publication 23 June 2022; date of current version 26 July 2022.

Digital Object Identifier 10.1109/TQE.2022.3184764

Quantum Volume in Practice: What Users Can Expect From NISQ Devices

ELIJAH PELOFSKE¹, ANDREAS BÄRTSCHI¹, AND STEPHAN EIDENBENZ¹

CCS-3 Information Sciences, Los Alamos National Laboratory, Los Alamos, NM 87544 USA

Corresponding author: Elijah Pelofske (e-mail: epelofske@lanl.gov).

This work was supported in part by the Laboratory Directed Research and Development Program of Los Alamos National Laboratory (LANL) under Project 20220656ER and in part by the LANL Information Science and Technology Institute. This work has been assigned LANL technical report number LA-UR-22-22058.

ABSTRACT Quantum volume (QV) has become the de-facto standard benchmark to quantify the capability of noisy intermediate-scale quantum (NISQ) devices. While QV values are often reported by NISQ providers for their systems, we perform our own series of QV calculations on 24 NISQ devices currently offered by IBM Q, IonQ, Rigetti, Oxford Quantum Circuits, and Quantinuum (formerly Honeywell). Our approach characterizes the performances that an advanced user of these NISQ devices can expect to achieve with a reasonable amount of optimization but without white-box access to the device. In particular, we compile QV circuits to standard gate sets of the vendor using compiler optimization routines where available, and we perform experiments across different qubit subsets. We find that running QV tests requires very significant compilation cycles, QV values achieved in our tests typically lag behind officially reported results and also depend significantly on the classical compilation effort invested.

INDEX TERMS Noisy intermediate-scale quantum (NISQ), NISQ benchmarking, qiskit, quantum computation, quantum volume (QV), quantum compilation.

I. INTRODUCTION

Quantum volume (QV) [19] has been designed as a benchmark measure for noisy intermediate-scale quantum (NISQ) devices. Informally speaking, an NISQ backend that has passed a QV protocol test of 2^m will largely correctly execute any quantum circuit on m qubits with up to m random 2-qubit gates on each of those qubits, thus giving a good guideline to users of the device as to what circuit sizes and depths appear reasonable to run on the device. We will give the more formal definition later. In this research, we aim to characterize the QV values of different NISQ backends as it would likely be experienced by regular, albeit somewhat sophisticated users of these systems.

QV has been defined to include the compilation from abstract circuit representation to the hardware connectivity and native gateset of NISQ devices. This is a necessary part of such a benchmarking definition; however, this also means that heavy circuit compiler optimization can be a very large factor impacting the computed QV measure. Our QV testing approach thus starts from a compiler-agnostic perspective. Specifically, all of the initial compiled circuits we send to the different backends are initially compiled using the Qiskit [5] transpiler when required, and otherwise directly submitted to the backend. However, once the circuits are sent

to the backend, further circuit optimization may occur. A user of NISQ [11], [48] quantum processing units (QPUs) will generally not have the tools, inclination, expertise, or time to perform heavy circuit compilation. Instead, they will use available open-source software. This is why we use the Qiskit [5] transpiler as an initial basis for comparison (not all systems allow the user to directly handle compilation of the quantum circuits).

Our main findings are as follows.

- 1) Preparation of QV circuits can be remarkably time intense depending on the system and not well standardized across different providers, even though QASM [20] is used by several vendors; there is no standardized way across vendors to turn a logical QV circuit, expressed in a standard circuit description language into an optimized, device-specific instruction sequence.
- 2) The qubit mapping and routing problem that quantum compilers address is quite difficult [39], [43]. The compilation toolchain within the software ecosystem and the backend itself greatly impacts circuit execution quality. Therefore, comparisons between QV values should also take into account the compilation method that was used. In particular, we find that using IBM Q's

custom QV compiler for IBM Q devices indeed improves QV values over a standard Qiskit compilation method.

- 3) QV values achieved by users typically lag officially reported values, and in some cases, the QV values of backends are not reported. The highest QV values we measured for each vendor are IBM at 16 using default transpilation and 32 when using more optimized compilation, Quantinuum (lower bound) at 512, IonQ at 8, Rigetti at 8, and Oxford Quantum Circuits (OQC) at $\neq 2$. These results are consistent with the backend error rates.
- 4) Providing a qubit subset choice for the logical to physical qubit mapping allows us to characterize the regions of the backend (i.e., the qubit and 2-qubit interacting gates) that give the highest fidelity QV results. Not all qubits and connections on an NISQ device are of the same quality; thus, even if a device passes a QV test, such success often relies on selecting a good qubit subset, which is not trivial to identify. This fact slightly compromises the original intuitive appeal of the QV measure: Passing a QV 2^m test does not necessarily imply that the device will generally handle any circuit of depth and width m well because it may not start with a good qubit subset choice.

Overall, we find that hardware vendors have made great progress in the past five years in their device quality. While even advanced users may not quite reach the officially reported QV values, they can expect to just lag a small factor behind. Nevertheless, our findings also point to the need for quantum circuit optimization for any practical application through advancing compilation tools: While we have had the opportunity to run an extensive amount of QV test runs, most quantum computing users should not have to go through such intense optimization procedures to test their algorithms.

This article is organized as follows. Following a literature review on the current state of QV research, in Section II, we summarize the methods and backends we will test. In particular, we differentiate the tests between *black-box* execution on all backends and then more customized compilation and execution on the IBM Q backends, including connected subgraph compilation and more heavily optimized compilation from a toolkit provided by IBM Q. Next, we present the results of this analysis across all 24 NISQ devices in Section III. We conclude with a discussion in Section IV about what these results show about the state of NISQ computers and the QV metric. The QV circuits as well as the corresponding measurement counts are publicly available [45]. Code is publicly available as well [44].

A. LITERATURE REVIEW

QV was proposed as a near-term metric for modern quantum computers that encompasses all aspects of the computational ability of the QPU including connectivity, qubit number, compiler software, and error rates [19]. Following this, more

advanced compiler and routing techniques [43] and Qiskit Pulse optimization [58] allowed measurement of QV 64 on some IBM Q systems [33]; we use this advanced implementation for some of our experiments.

On Quantinuum (formerly Honeywell) H1 backends, the measured QV has been steadily increasing along with new system upgrades and lower error rates, going from 64 [47], to 1024 [9], 2048 [51] and, finally, the maximally possible $4096 = 2^{12}$ [52] on the 12-qubit **System Model H1-2** Quantinuum device.

There are several other NISQ hardware-independent benchmarks similar to QV that have been proposed, including mirror circuits [49], application-oriented benchmarks [40], [41], volumetric benchmarks [12], which are generalizations of the QV benchmark, and machine learning motivated metrics such as the proposed q-BAS score [10]. Using six scalable application-oriented benchmarks, numerous devices (across IBM Q, Rigetti, and IonQ) have been benchmarked [16]. For another NISQ benchmark, Atos has proposed a metric called *Q-score* as an application-relevant benchmark [8]. One of the other key computation metrics that need to be established for NISQ computers, as it was with classical computers, is a notation of *speed*. To this end, Circuit Layer Operations per Second (CLOPS) has been proposed as a viable NISQ computer speed metric [59].

A recent interesting topic is applying error mitigation to the QV protocol in order to demonstrate an increased effective QV [38].

II. METHODS

A QV circuit consists of sequences of random qubit index permutations followed by a layer of random two-qubit unitaries from *Special Unitary* matrices of degree 4 **SU(4)**. Compiling layers of SU(4) gates with arbitrary connectivity between qubits is especially challenging for compilers [61] in large part due to the hardness of circuit mapping and qubit routing [17] on restricted connectivities. Given in [19], a generic square QV circuit with depth d and width m is a sequence of d circuit layers

$$U = U^{(d)} \dots U^{(2)} U^{(1)} \quad (1)$$

where each of these layers (e.g., $U^{(d)}$) is of the form

$$U^{(t)} = U_{\pi_t(2m'-1), \pi_t(2m')}^{(t, m')} \otimes \dots \otimes U_{\pi_t(1), \pi_t(2)}^{(t, 1)} \quad (2)$$

indexed by t ranging from 1 to d . Each layer is acting on $m' := \lfloor \frac{m}{2} \rfloor$ pairs of qubits (meaning that if m is odd then one qubit in each layer will be idle). Each layer is created by choosing a uniform random permutation π_t of the m qubit indices, and then applying the two-qubit unitary gates $U_{\pi_t(2i-1), \pi_t(2i)}^{(t, i)}$ from **SU(4)** acting on qubits $\pi_t(2i-1)$ and $\pi_t(2i)$ for all m' pairs of qubits being used in this layer. Fig. 1 is an example diagram of the logical structure of a QV circuit.

For a given QV circuit, the relevant question is how well the quantum device implemented and executed the circuit [19]. To this end, the QV protocol uses the *heavy output*

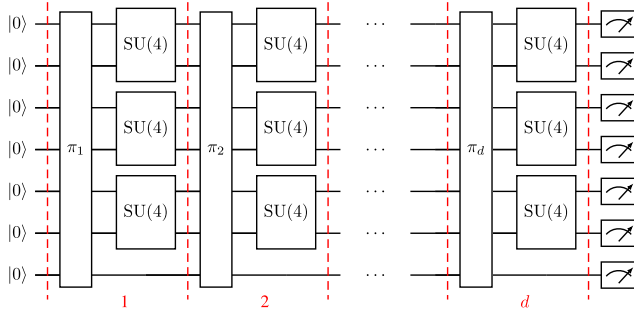


FIGURE 1. Quantikz [36] circuit drawing for a Logical QV circuit on $m = 7$ qubits, where each layer $1, \dots, d = 7$ consists of a random permutation of the qubit labels followed by $m' = 3 = \lfloor \frac{7}{2} \rfloor$ random $SU(4)$ gates acting on consecutive pairs of (the permuted) qubits.

generation problem [2]. Given a QV circuit U , it will have an ideal bitstring output distribution of

$$p_U(x) = |\langle x|U|0 \rangle|^2 \quad (3)$$

where x is a bitstring output with length equal to m . The central idea of the heavy output generation problem is to partition all possible observable bitstrings into two equal-sized sets; one of which has a lower ideal output probability, and one of which has a higher ideal output probability (i.e., the set of heavy outputs). The quantum hardware implementation of U is then considered successful if more than two-thirds of the measured output bitstrings fall into the heavy output set. More formally, given the full ideal probability distribution $p_U(x)$, we sort each probability such that $p_1 \leq p_2 \leq \dots \leq p_{2^m}$. Then, we can partition according to the median of the probabilities p_{median} in order to get the heavy output set of bitstrings for U

$$H_U = \{x \in \{0, 1\}^m \text{ such that } p_U(x) > p_{\text{median}}\}. \quad (4)$$

Thus, the *heavy output probability* (HOP) for a quantum circuit U implemented on a backend is defined as the number of heavy bitstrings found in the distribution (i.e., the number of elements in H_U) out of the total number of samples taken on the backend. This measurement is then repeated for multiple QV circuits (say k distinct QV circuits, each with their own random permutations and random seeds) in order to determine if the quantum backend in question can reliably sample HOP distributions with a probability greater than two-thirds. In the limit of the number of QV circuits (and for large circuit size m and depth d values), the expected mean HOP approaches $\frac{1+\ln(2)}{2} \approx 0.85$ [2], [19]. For k distinct QV circuits, each circuit QV_i has a measured HOP denoted as $\text{HOP}(QV_i)$. Treating the HOP outcome as a binomial distribution (i.e., either the backend passes two-thirds or it fails to pass two-thirds, over many circuits, this can be approximated as a normal distribution. Using this approximation, we can compute confidence intervals on the resulting distribution: Equations (5) and (6) show the formulas for computing the mean HOP, then the standard deviation of the distribution, and then (7) and (8) show the formulas for computing the 99% confidence interval. These statistical tests are important

because they show when a particular distribution of HOPs is above the two-thirds threshold with a high confidence level

$$\text{mean} = \frac{\sum_i^k \text{HOP}(QV_i)}{k} \quad (5)$$

$$\sigma = \text{mean} \cdot \sqrt{\frac{(1 - \text{mean})}{k}} \quad (6)$$

$$z = \frac{(\text{mean} - \frac{2}{3})}{\sigma} \quad (7)$$

$$Z_{\text{conf}} = 0.5 \cdot \left(1 + \text{erf} \left(\frac{z}{\sqrt{2}} \right) \right). \quad (8)$$

In order to determine if a device (or qubit subset of the device) passed the QV protocol, we use the following criteria.

- 1) The mean of the HOP states is above two-thirds [see (5)].
- 2) 2σ below the mean of the HOP state probability is also above two-thirds [see (6)].
- 3) The distribution is above two-thirds with a 0.99 z-confidence interval [see (7) and (8)].

These requirements increase in strictness; 2σ corresponds to a z-confidence of 0.977 (which has a corresponding to z value of 2), compared to the 0.99 z-confidence. The QV metric was originally defined on a quantum computer with at least m qubits [19] and $d(m)$ denoting the largest d such that criteria 1) and 2) are satisfied

$$\log_2 QV = \arg \max_m \min(m, d(m)). \quad (9)$$

When trying to find the QV, we can restrict ourselves to tests with $d = m$. For simplicity, in the remainder of the article, we will denote $n = \log_2 QV$, and $n = m = d$.

By this definition, the best QV value found on a backend will be the QV value of that backend. In particular, this means that distinctions between different qubit subsets have not been specifically reported when applying the QV protocol. One of the methods we investigate is distinguishing how the QV circuits perform on specific qubit subsets, as opposed to simply taking the best performing value found on the device (see Section II-C). However, this is not possible on all devices because some systems do not currently support specifying a qubit subset to compile the logical circuit to.

In order to standardize the circuits used on all backends, 1000 QV circuits per QV circuit size are generated using the Qiskit QV method [32]. Fig. 4 shows the gate counts for the raw uncompiled circuits.

Table 1 summarizes the details and published hardware metrics of the 24 NISQ backends we test. Table 1 also summarizes the QV values we found on each of the 24 backends. These values are differentiated between 1) the QV value found when using the black-box execution method (little or no qubit mapping or basis gate conversions), and 2) the best QV value found across all circuits executed on that backend that were compiled using more time intensive compilation

TABLE I. Table of NISQ QPUs Evaluated Using the QV Protocol

Vendor	Backend name	Measured QV			QPU			Mean fidelity		
		Black-box	Argmax	Vendor	# Qubits	Type/Topology	#Edges	2Q gate	1Q gate	SPAM
Quantinuum	H1-2	512*	512*	4096	12	All-to-All	66	0.995	0.9997	0.993
IBM Q	ibmq_lima	8	8	8	5	Falcon r4T	4	0.9898	0.9998	0.973
	ibmq_belem	8	8	16	5	Falcon r4T	4	0.9874	0.9998	0.9775
	ibmq_quito	8	16	16	5	Falcon r4T	4	0.9889	0.9998	0.9717
	ibmq_jakarta	8	8	16	7	Falcon r5.11H	6	0.9896	0.9997	0.9747
	ibmq_manila	16	16	32	5	Falcon r5.11L	4	0.9897	0.9997	0.9728
	ibmq_bogota	8	8	32	5	Falcon r4L	4	0.9905	0.9998	0.9656
	ibmq_perth	8	8	32	7	Falcon r5.11H	6	0.9781	0.9997	0.987
	ibmq_casablanca	8	16	32	7	Falcon r4H	6	0.9903	0.9998	0.9805
	ibmq_lagos	8	32	32	7	Falcon r5.11H	6	0.9924	0.9998	0.9862
	ibmq_guadalupe	8	32	32	16	Falcon r4P	16	0.9892	0.9997	0.9738
	ibmq_sydney	8	16	32	27	Falcon r4	28	0.9487	0.9997	0.957
	ibmq_toronto	8	16	32	27	Falcon r4	28	0.9787	0.9993	0.9376
	ibmq_brooklyn	8	32	32	65	Hummingbird r2	72	0.9118	0.9995	0.9694
	ibmq_washington	8	16	64	127	Eagle r1	142	0.9828	0.9997	0.9737
	ibmq_auckland	8	16	64	27	Falcon r5.11	28	0.9536	0.9992	0.9872
	ibmq_cairo	8	16	64	27	Falcon r5.11	28	0.9882	0.9998	0.9807
	ibmq_hanoi	8	32	64	27	Falcon r5.11	28	0.9891	0.9998	0.9778
	ibmq_mumbai	8	16	128	27	Falcon r5.1	28	0.9504	0.9994	0.972
	ibmq_montreal	8	32	128	27	Falcon r4	28	0.9858	0.9996	0.9769
IonQ	Harmony	8	8		11	All-to-All	55	0.96541	0.9972	0.99709
Oxford Quantum Circuits (OQC)	Lucy	< 2	< 2		8	LNN ring	8	0.9416	0.9991	0.9044
Rigetti	Aspen-11	< 2	4	8	38	Octagonal	43	0.9215	0.9955	0.9678
	Aspen-M-1	< 2	8	8	80	Octagonal	102	0.9113	0.9894	0.9695

The values in the *Black-Box Measured QV* Column are what users who execute circuits without tuning can expect, whereas the values in the *Argmax Measured QV* column are the maximum values we were able to validate with significant additional effort as described in Sections II-B, II-C, and II-D. Values in the *Vendor Measured QV*, as well as single-qubit *1Q Gate*, two-qubit *2Q Gate*, and state preparation and measurement *SPAM* fidelities, are all vendor-provided metrics. The mean gate and SPAM fidelities are computed across all operations of the same type available on the device during the whole qv circuit execution duration (which did span several weeks in some cases). The Rigetti device fidelities were computed using the nonsimultaneous gate operation vendor-provided calibration data. The IBM Q single-qubit error rate averages include the zero error rate **Rz** gate, and not including **Id** gate error rates. The number of edges for each backend was counted simply as the number of connections between qubits; this does not count bi-direction gate operations, or multiple different gate operations between two qubits. *The QV value for the **H1-2** device is a lower bound only, not the finally measured QV value; the device has passed all QV experiments up to QV 512 and larger QV circuits are not yet completed.

procedures (these heavier compilation procedures are specific to IBM Q and Rigetti backends, see Sections II-B, II-C, and II-D). The procedure on all backends is to start at a small QV circuit size (e.g., $n = 3$) and then iterate to either larger circuit sizes or smaller circuit size depending on the results of the initial test. Once the device clearly fails to pass at a given n , the procedure is terminated. However, this procedure has not fully completed for the **H1-2** Quantinuum backend due to usage limitations, so far passing experiments up to $n = 9$. Therefore, its reported value of QV 512 is a lower bound on the true (black-box) QV of the device.

Next we will detail the full QV circuit compilation procedure, a high-level diagram of which is shown in Fig. 3. Across all backends (except the Quantinuum backend), we use the Qiskit transpiler method [5] to take the QV circuits and compile them to a specific gateset and a specific qubit layout. The gateset supplied for compilation corresponds to the native gateset that is supported by the backend to the best of our knowledge. No other information (e.g., gate execution times, error rates, etc.) is supplied to the transpiler. The software versions we use are **qiskit=0.33.1**, **qiskit-terra=0.19.1**, and **amazon-braket-sdk=1.9.5**, up to **amazon-braket-sdk=1.21.0**.

When submitting jobs through the Amazon Braket SDK (this includes the backend providers IonQ, Rigetti, and OQC), there is a small but important difference between the Amazon Braket SDK and Qiskit; Amazon Braket does not have a measure gate. Instead, the measure gates are applied implicitly to qubits which had gates applied. This becomes important in the $n = 3$ QV circuit case, where it is possible to create a QV circuit where all three layers are only acting on two qubits throughout the entire QV circuit; resulting in an idle qubit. In the Qiskit implementation, this qubit was still measured (even though no gate operations had been applied to it). Therefore, to maintain consistency with Qiskit and to not encounter noncontiguous qubit indexing errors, in the Amazon Braket implementation a single identity gate was applied to the idle qubit.

The IBMQ backends allow *jobs* to be submitted to the backend, where a job is a collection of circuits. However, there are constraints on the number of circuits (and the number of samples) that can be used per job. For all IBMQ experiments, we submit 4 jobs each composed of 250 QV circuits in order to execute each sequence of 1000 QV circuits. Thus, while these jobs are submitted in sequence, there is possibly a time delay in between these four blocks of circuits.

Figures in this article were generated using Qiskit [5] and Matplotlib [14], [26].

A. BLACK-BOX QV

The simplest method we use is to simply submit the uncompiled circuits to the specified backend (and let the backend or system handle compilation). However, how this method is implemented varies depending on the backend. In some cases, directly submitting the uncompiled circuits to the backend is not possible because the gateset is incompatible or the software is incompatible, therefore requiring custom code to be developed to handle this conversion (for example, in the case of the backends provided through Amazon Braket). In other cases, the system allows for very quick and direct submission of the uncompiled circuits (for example in the case of Quantinuum). We give details in the following sections for each hardware vendor.

1) QUANTINUUM

The black-box method to access the **H1-2** Quantinuum backend is simple: The Honeywell API allows users to submit QASM [20] code to each of these backends with no initial compilation needed.

In accordance with the black-box execution approach, we do not compile or optimize these circuits at all before submitting them to the backend via the provided Quantinuum Python API. The API allows the optional **no-opt** compiler flag to be specified, which for the black-box approach we set to the default (which is **False**), which allows the backend to perform compiler optimizations. The current published QV value of the **H1-2** system is 4096 [52] (increased from the previous 2048 [51]).

Our access to the Quantinuum backend was granted through Oak Ridge National Laboratories OLCF program.

2) BLACK-BOX IBM Q

For the IBM Q backends, the black-box method we use is to call the Qiskit **execute** method [5], [28], [31] using the flags **optimization_level=3** and **layout_method=noise_adaptive**. Although not specified by the user, **sx**, **rz**, **cx**, **x** are the IBM Q basis gates. This method compiles the original QV circuits onto that backend. Note that the Qiskit transpiler does not always successfully compile a group of circuits; the transpiler exits after unsuccessfully compiling for 1000 iterations. Therefore, for a group of circuits we run the **execute** method until it successfully compiles. Overall, this makes the method not real-time efficient.

3) IONQ

The 11-qubit IonQ backend was accessed through Amazon Braket. In order to submit jobs through this service, the circuits need to be specified using the supported gates. In order to help the compilation, we compile the QV circuits to the IonQ gateset. The compiler we use in this stage, as with the

other backends, is the Qiskit [5] transpiler. Here, the Qiskit gateset we use for compilation is **rx**, **ry**, **rz**, **rx**. Once converted to QASM, we convert the Qiskit gateset to a supported Amazon Braket gateset; **xx**, **ry**, **rz**, **yx** (with **XX** being a native two-qubit gate supported by the IonQ backend [24]). These gates can then be converted to Amazon Braket SDK code and submitted to the IonQ backend. However, the circuit that is compiled and run on the backend is not visible to the user.

IonQ has not published the QV of the 11-qubit trapped-ion quantum computer Harmony, available through Amazon Braket [40]. However, there has been application-oriented benchmarking of the 11-qubit device [60].

4) OQC

OQCs backend **Lucy** [50] can be accessed through Amazon Braket. Using the Qiskit transpiler, the QV circuits were compiled using the unidirectional linear-nearest-neighbors (LNN) ring gate connectivity, optimization level 3, and basis gates **rz**, **sx**, **x**, and **ECR** (these are the basis gates for the OQC Lucy backend). Then, these circuits are converted into Amazon Braket syntax and submitted to the backend. As with the other Amazon Braket available backends, we allow the compilation software stacks of Amazon Braket and OQC handle all aspects of compilation and qubit assignment. One of the options the user has when submitted circuits to the OQC Lucy backend via Amazon Braket is to specify whether to turn OFF all circuit optimizations (this is accomplished in the Amazon Braket case by creating the circuit with a *verbatim_box* option) or to leave on all circuit optimizations. By default, we leave on all circuit optimizations, which allows the OQC compiler to select which qubits to use and to optimize the circuit (Fig. 2 circuit (10) shows what one of these optimized circuits looks like, having been returned from the backend).

5) BLACK-BOX RIGETTI

The route we took for the black-box compilation on Rigetti devices was by accessing the **Aspen-11** and **Aspen-M-1** devices through Amazon Braket. The Qiskit gateset we use for compilation is **cz**, **rz**, **rx**. Once compiled to QASM, we convert the Qiskit gateset to a supported Amazon Braket gateset; **cz**, **rz**, **rx**. Note that **XY** is also a native gate of the Rigetti devices [3], however it is not currently a supported gate for the Qiskit transpiler. Additionally **CPHASE** is another native gate for the Rigetti backends; however, it was not used by the Qiskit to implement the QV circuits in this compilation route. These gates (**CPHASE**, **XY**, **cz**, **rz**, **rx**) form the native gateset of the **Aspen-11** and **Aspen-M-1** devices [35]. Once each circuit is represented in the Amazon Braket compatible gateset, it is submitted to the Rigetti backends. From there, the Rigetti QUILC compiler [56] compiles the supplied circuit to the backend connectivity based on the latest calibration data of the backend. The resulting compiled circuit is sent back to the user as Quil code, allowing us to analyze the circuit that

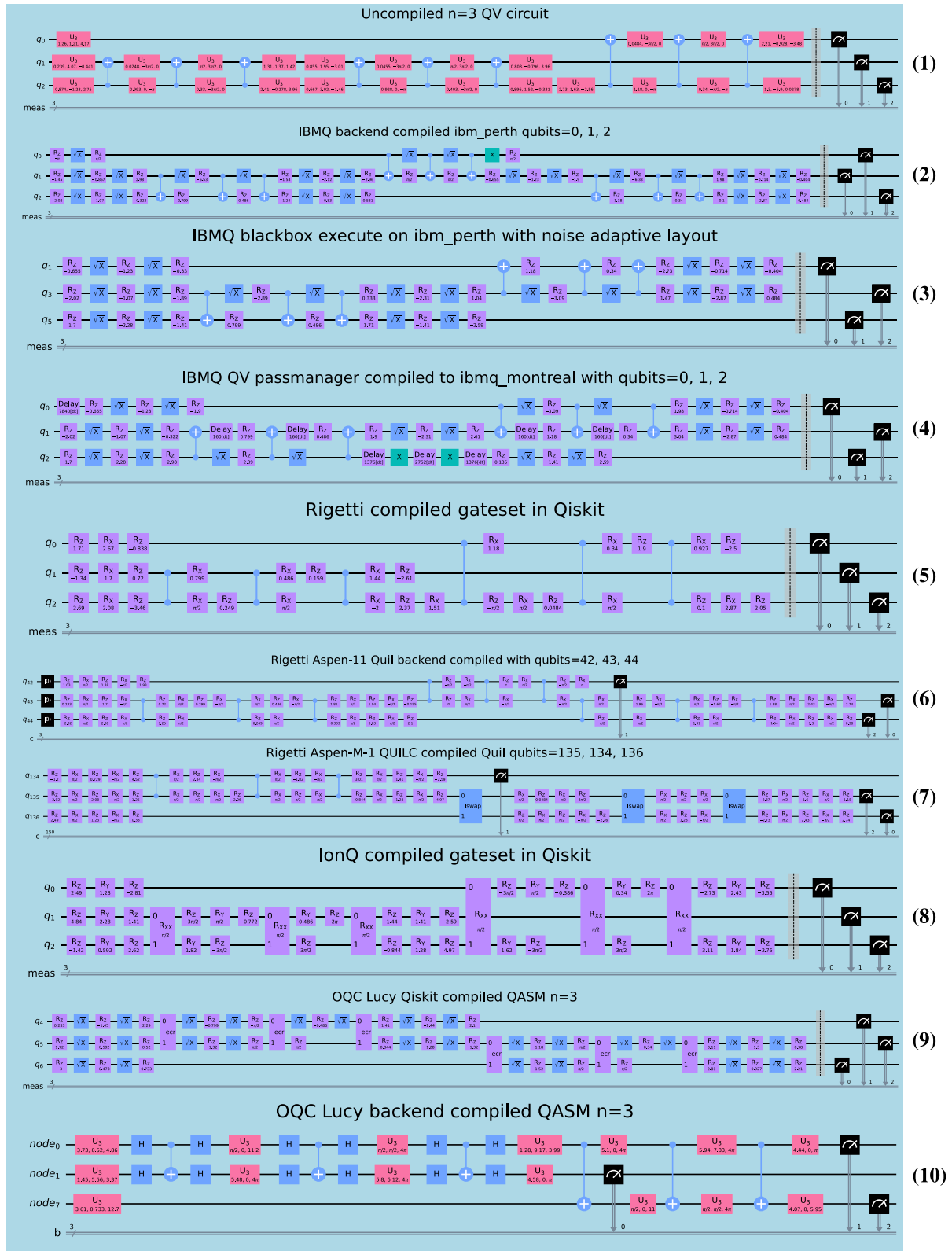


FIGURE 2. Impact of compilers: Starting from the original QV circuit (top) defined in Qiskit using `u3` and `cx` gates, we compile to backends with different connectivities, gatesets, and software. We use a simple $QV = 8$ ($n = 3$) circuit for illustration purposes. Note that the backend compiled circuits for IonQ and Quantinuum are not shown; these backends do not currently support returning the backend executed circuits to users.



FIGURE 3. High-level compilation procedure diagram.

was executed on the backend [6], [35], [54], [55]. In other words, for the black-box approach, we let the compilation stacks of Amazon Braket and Rigetti handle compilation and qubit assignment.

The QV of the **Aspen-11** and the **Aspen-M-1** have both been measured to be 8 [15], and previous Rigetti Aspen

devices (specifically Aspen-4) have had a measured QV of 8 [35] as well.

B. RIGETTI QUBIT SUBSET ENUMERATION

For a more detailed analysis of Rigetti Aspen-11 and Aspen-M-1 devices, we used the Rigetti Cloud Computing Services

TABLE II. Successful QV Experiments Across the IBM Q Backends Compared to the Vendor-Provided $\log_2$ QV

ibmq device	lima	belem	quito	jakarta	bogota	manila	lagos	perth	casablanca	guadalupe	sydney	toronto	brooklyn	hanoi	cairo	mumbai	montreal	auckland	washington
# qubits	5	5	5	7	5	5	7	7	7	16	27	27	65	27	27	27	27	27	127
IBM $\log_2$ QV	3	4	4	4	5	5	5	5	5	5	5	5	5	6	6	7	7	6	6
$n = 3$	4/4	2/4	4/4	7/7	3/3	3/3	7/7	4/7	7/7	17/20	23/27	26/37	80/95	30/37	34/37	31/37	31/37	18/37	
$n = 4$	0/3	0/3	1/3	0/6	0/2	0/2	2/6	0/6	2/6	4/24	5/48	9/48	22/132	9/48	12/48	13/48	14/48	3/48	28/262*
$n = 5$	0/1	0/1	0/1	0/6	0/1	0/1	0/6	0/6	0/6	0/30	0/68	0/68	0/200	0/68	0/68	0/68	0/68	0/68	

Denominator is the number of connected subgraphs of size n on the backend, numerator is the the number of those subgraphs that passed the QV protocol test. *On **Ibm_washington**, in part due to the significantly larger chip size than the other IBM Q backends, not all circuits were able to be run; the true number of connected qubit subsets on **Ibm_washington** is 272, but we only tested 262 of those.

(QCS) platform in order to perform qubit subset enumeration for the QV circuits. Fig. 3 shows the data pipeline for this compilation; starting at the uncompiled QV QASM circuits, we employ the Qiskit transpiler to convert the circuits into a connectivity matching a qubit subset of either Aspen-11 or Aspen-M-1 using the gateset of rz, rx, and cx. Next, this representation was converted into the Quil instruction set [55], and then, the Rigetti QUILC compiler [56] was called in order to compile the Quil program into a device compatible form. Finally, the circuit was submitted to the device to be executed (using 100 samples).

One important distinction between this method and the black-box Rigetti approach described in Section II-A5 is the use of *Active Reset* [35]. Active Reset decreases the amount of computation time used on the device by speeding up the process of qubit reset; the cost is a decrease in qubit fidelity. For the Aspen-11 and Aspen-M-1, the Active Reset fidelities, based on vendor-provided calibration data on the QCS platform are typically approximately 99%. The Amazon Braket execution process (i.e., the black-box approach of Section II-A5) always uses Active Reset. By contrast, in the case of the qubit subset enumeration done on the QCS platform, we do not use the Active Reset capability. This difference can be directly seen in Fig. 2; circuit (6) was submitted through Amazon Braket and it specifies Active Reset for the three qubits at the start of the circuit, whereas circuit (7) was compiled using QUILC for the purpose of the qubit subset enumeration procedure, and no active reset is used in that circuit.

C. QISKIT TRANSPILER WITH QUBIT SUBSET ENUMERATION: IBM Q

In order to more thoroughly test QV values achievable, we specify which groups of qubits to compile the QV circuits to, instead of letting the compiler (local or backend) handle this. This is specified using the *initial layout* parameter in the Qiskit transpiler.

The Qiskit gateset we use for compilation is **sx, rz, cx, x**. We compile each of the 1000 QV circuits onto each of the connected subgraphs of each of the IBM Q backends. We learned that this compilation process is quite time intensive

and requires HPC multiprocessing. We spent approximately 100 000 CPU h (including the custom QV passmanager compilation time) to compile the $n = 3$ through $n = 7$ circuits onto 19 different IBM Q backends (see Table 2). The main reason for the enormous amount of compile time required is that the compiler often reaches a *maximum 1000 iteration error*, thus possibly requiring many attempts to compile a given circuit onto a connected subgraph of the hardware. Although this is an allowed component of the QV protocol, it is not time efficient for users; the black-box compilation (see Section II-A2) is closer to what a typical user would implement.

The arguments we use for the transpiler method are the **coupling_map** of the backend, **optimization_level=3**, **initial_layout** of the connected subgraph, and **basis_gates=x, sx, cx, rz**. Occasionally, the Qiskit transpiler will use some neighboring qubits outside of this connected subgraph in an attempt to improve circuit fidelity. Based on the compilation described in [19], this is an accepted part of the definition for QV. One of the causes of the transpiler reaching a *maximum 1000 iteration error* is because of a qubit subset choice that is poorly chosen. For this procedure, we do not care about the order of the qubit subset choice (because the order of the qubits used in the circuit can be remapped for different connectivities); therefore, we also randomly shuffle the qubit subset while attempting to compile each circuit.

D. CUSTOM COMPILATION USING QV PASSMANAGER: IBM Q

Finally, we use the custom QV compiler techniques introduced in [33], [43], and [58] in order to compare how heavier compilation affects the measured QV compared to using the standard Qiskit transpiler. The software used in these experiments is published on the *qiskit-tutorials* Github [29], [30]. Specifically, we implement the same qubit subset compilation of Section II-C, except we now use the custom QV compiler [29], [30]. In particular, we attempt to compile a sorted qubit subset and then several random permutations of the qubit subset. However, some circuits fail to compile to some qubit subsets using the custom QV compiler. The passmanager flags are all set to the default, in particular these are flags we specify: **basis_gates**, **coupling_map**,

`qubit_subset`, `backend_props=backend.properties()`, `instruction_durations` (provided from the backend), `synthesis_fidelity=.99`, `pulse_optimize=True`.

This method requires even more computation time than the compilation with qubit subset enumeration from Section II-C. The routing and qubit assignment optimization [33], [43] is done with the CPLEX solver [18]; meaning that this method requires a CPLEX license to compile these circuits. For all compilation we set the **BIPMapping** (i.e., the CPLEX optimization of routing and qubit assignment) timeout to 5000 s [27]. This custom transpilation also uses Qiskit Pulse [4], [58] optimization to increase circuit fidelity. The use of Qiskit Pulse additionally requires precise timing of the gate instructions, which is enforced by delay gates in the circuits. An example of the usage of these delay gates can be seen in Fig. 2 circuit (4).

Additionally, we cut off compilation of all of the circuits after a few days using HPC resources; in some cases, there were circuits (and qubit subset choices) that were not attempted to be compiled because of the time constraint we set. Therefore, either because of compiler errors preventing compilation, or because of the time constraint we imposed, not all circuits for all qubit subsets could be compiled across the backends we tested using this compilation method. We also restricted these compilations to a subset of the available IBM Q backends, as opposed to Section II-C where we compiled circuits for all available IBM Q backends.

III. RESULTS

We compile and execute the QV circuits on the backends listed in Table 1. First, we show results for black-box compilation and execution, which is the most general and widely available method across the backends (for example, not all backends allow full specification of which qubits to use in the circuit). Next, we show results for IBM Q backends when we enumerate compilation across the qubit subsets (of size n) of a backend. This allows us to characterize the QV protocol results across the entire chip of the IBM Q backends. Finally, we evaluate the **custom IBM Q VQ passmanager** for circuit compilation [29], [30], [33] on a restricted set of IBM Q backends and connectivities on those backends. Unless otherwise noted, all experiments used 100 samples for each circuit execution.

Fig. 2 show the differences in circuit compilation starting from the original uncompiled circuit to the compiled circuits that are submitted to the different devices. In order from the top to bottom circuit drawings in Fig. 2, following can be seen.

- 1) Uncompiled QV circuit; all subsequent circuits use this description to compile to the other gatesets or connectivities.
- 2) Qiskit transpiled circuit on qubits 0, 1, 2 on `ibm_perth` with optimization level 3. Corresponding methodology described in Section II-C.

- 3) Transpiled using the black-box **execute** method on `ibm_perth`. Corresponding methodology described in Section II-A2.
- 4) Compiled using the custom IBM Q VQ passmanager with pulse level optimization. Corresponding methodology described in Section II-D.
- 5) Qiskit compiled circuit for the Rigetti Aspen-11 backend that was submitted to Amazon Braket for the black-box Rigetti method. Corresponding Methodology described in Section II-A5.
- 6) Compiled Quil circuit that was executed on the Rigetti Aspen-11 backend having been submitted through Amazon Braket. Corresponding methodology described in Section II-A5. The active reset instructions on all three qubits can be seen at the start of the circuit.
- 7) QUILC compiled circuit on Aspen-M-1 for the qubit subset enumeration method. This circuit used native XY two-qubit gates, but in this Qiskit circuit drawing, we use `iSWAP` gate in place of the $XY(\pi)$ gates since those two are equivalent. Corresponding methodology described in Section II-B.
- 8) Qiskit compiled circuit that was submitted to the IonQ Harmony backend (having been converted into Amazon Braket python code). Corresponding methodology described in Section II-A3.
- 9) Qiskit compiled circuit to the Lucy OQC gateset and unidirectional connectivity that was submitted to the backend via Amazon Braket. Corresponding methodology described in Section II-A4.
- 10) OQC Lucy backend compiled circuit that was returned with the job metadata. Corresponding methodology described in Section II-A4.

In Fig. 2, the difference in structure that results from the same logical $n = 3$ circuit is perhaps surprising, even though some of the diversity can be explained with the different native gate sets that the compilers aim to optimize to. Interestingly, when comparing circuit (6) and circuit (5) in Fig. 2, we see that although the supplied QV circuit used 6 two-qubit `cz` gates, the software stack that compiled the circuits chose to use a total of 9 `cz` gates in the final Quil compiled circuit, which is suboptimal. Note that although the OQC Lucy backend native gateset uses `ECR` as the two-qubit gate, the returned QASM code that was compiled for the backend [circuit (10)] used `CNOT` gates for two-qubit interactions.

Fig. 4 show the average circuit statistics in terms of gate depth, one-qubit gate counts, and two-qubit gate counts, across the different device compiled circuits. Across different QV circuit sizes, the **custom QV passmanager** compilation reduces the `CNOT` count on average compared to the two other IBM Q compilation procedures.

In order to visualize how the QV protocol progresses as we execute each circuit on the given QPU, we show cumulative HOP figures where the x -axis is the circuit index, and the y -axis is the HOP. In these figures, we plot the ideal HOP

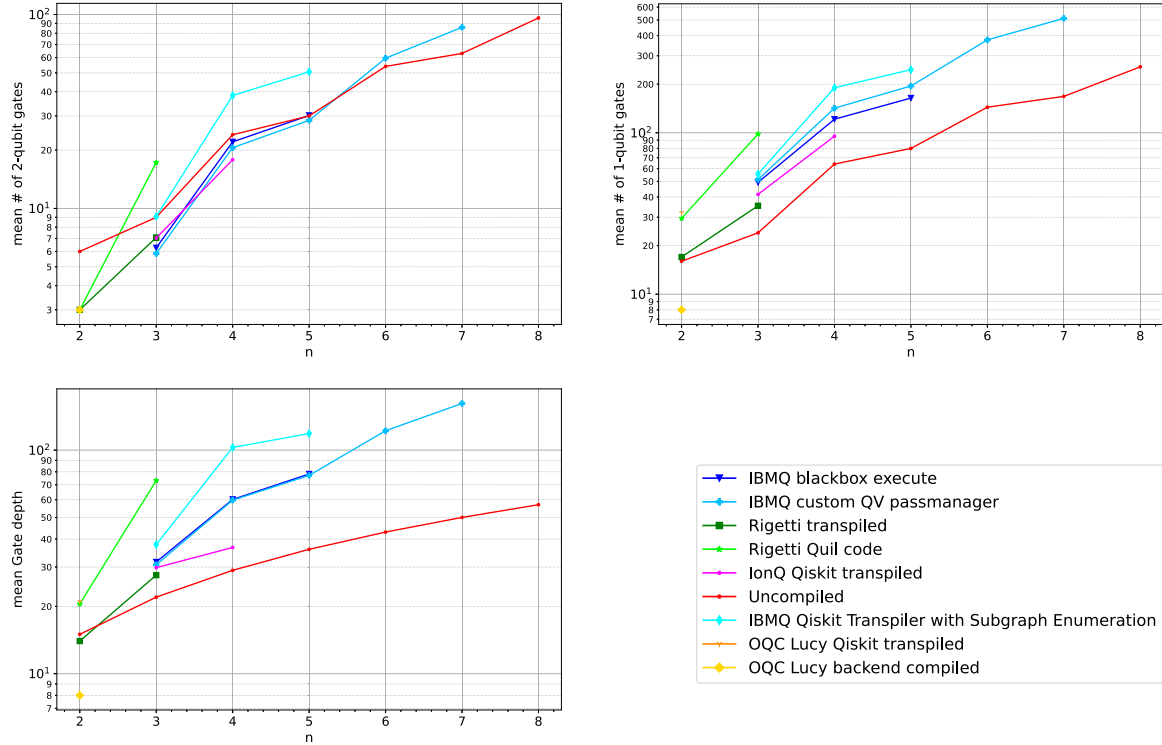


FIGURE 4. QV circuit operations summary figures. Average single-qubit operations (upper right), two-qubit operations (upper left), SPAM operations (bottom left), and the legend (bottom right). The two-qubit gate counts were computed as follows; Qiskit transpiled code to the Rigetti gateset: `cz`, Rigetti Quil backend black-box compiled code: `cz`, `XY` (the CPHASE gate was never introduced into the circuits by the QUILC compiler during black-box execution), IonQ Qiskit transpiled code: `RXX`, IBM Q backend compiled code: `cx`, uncompiled code: `cx`, Qiskit compiled to OQC Lucy gateset: `ECR`, OQC Lucy backend compiled circuit: `cx`. The single-qubit gate counts were computed as follows; Qiskit transpiled code to the Rigetti gateset: `rx`, `rz`, Rigetti Quil backend compiled code: `rx`, `rz`, IonQ Qiskit transpiled code: `rz`, `rx`, `ry`, IBM Q backend compiled code: `rz`, `x`, `sx`, uncompiled code: `U3`, Qiskit compiled to OQC Lucy gateset: `sx`, `x`, `rz`, OQC Lucy backend compiled circuit: `U3`. See the Appendix A for details on the gate definitions. Note that the *delay* gates in the pulse optimized IBM Q circuits are *not* counted in the gate depth or single-qubit gate counts. Measurement operations are also not counted in the single-qubit counts. All plots have a log y-axis scale.

distribution, the individual circuit measured HOP values, the cumulative mean of the HOP values (up to index i in the plot), 2σ below the cumulative HOP mean, and finally, we color shade the region with z-confidence > 0.99 if more circuits are executed past that confidence level (this can be seen in Figs. 6, 11, and 14). We encourage close attention to the x -axis whenever comparing HOP plots: While we usually plot up to the full 1000 circuits, we cut off earlier when it is clear that the test has been passed.

In all HOP figures, we plot both the mean HOP (solid orange horizontal line), as well as the individual ideal HOP values for each circuit (shown as high transparency orange points). Note that the 1000 QV circuits have smaller ideal HOP at $n = 2$ and $n = 3$ compared to larger values of n . This can be seen in the ideal distributions of Fig. 9 for $n = 2$ compared to $n = 5$ or greater plots (for example, Fig. 5). This is to be expected for smaller QV circuit sizes [9], even though in the limit the ideal HOP distribution approaches $\frac{1+\ln(2)}{2}$.

A. BLACK-BOX QUANTINUUM

As described in Section II-A1, the QV circuits were submitted directly to the backend as the uncompiled QASM file (the circuit statistics on the uncompiled QV circuits can

be seen in Fig. 4), which is entirely comprised of CNOT and U3 gates (see Fig. 2). No user side circuit optimization, basis gate conversions, or transpilation was performed on these circuits.

Fig. 5 shows that the **H1-2** device passes the QV test for circuit sizes up to $n = 8$. In order to save resources, as with IonQ, execution was terminated once the QV protocol criteria were met. Due to usage constraints, larger circuit sizes are still being tested. Therefore, for the **H1-2** backend, we can only provide a lower bound ($n = 9$) on the QV value of the QPU. The $n = 5$ experiments used 100 shots for each circuit. The $n = 6, 8, 9$ experiments used 20 shots for each circuit. As a comparison, $n = 8$ was reached after only 140 circuits, which is considerably shorter than in particular most IBM took to reach some of their best results.

B. BLACK-BOX IBM Q

The QV results were nearly identical across all IBM Q backends when using the black-box **execute** method. That is, every backend passed the $n = 3$ QV test, and failed $n = 4$ and $n = 5$ with the exception of a particular qubit subset of **ibmq_manila** for $n = 4$. Table 1 summarizes these results. Fig. 6 shows the results where **ibmq_manila** passed for $n = 4$ sized QV circuits, but failed to pass at $n = 5$.

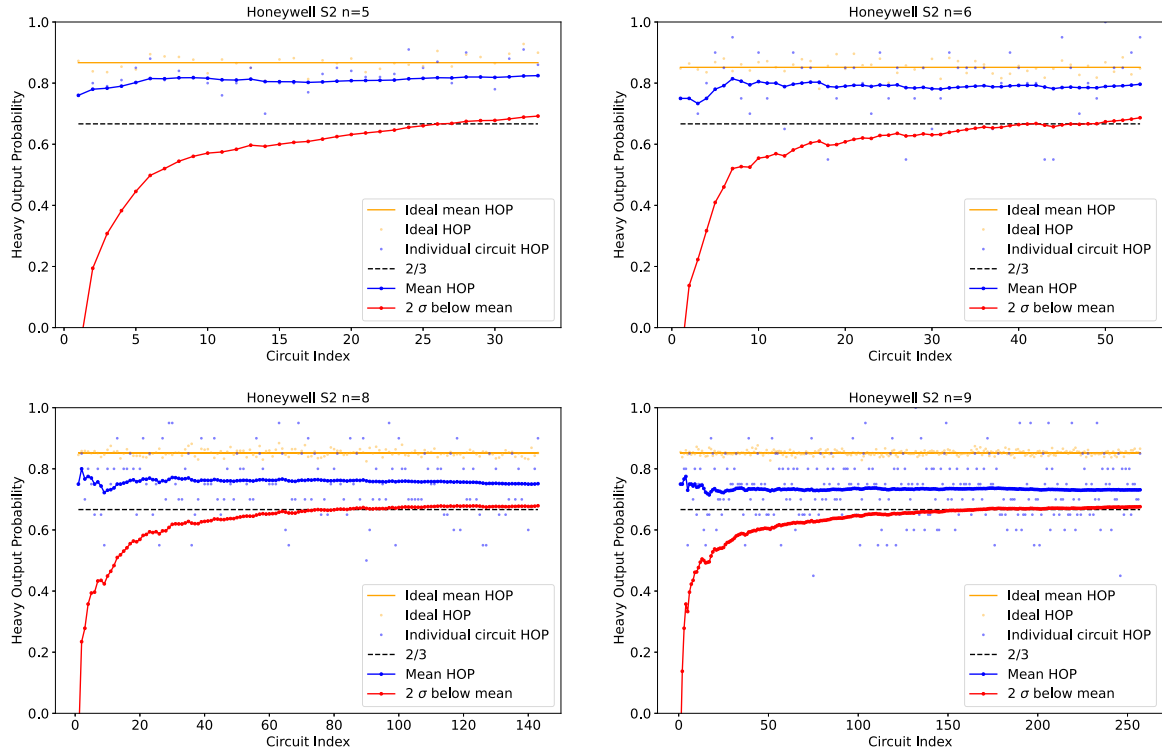


FIGURE 5. HOPs as a function of circuit index for the Quantinuum (formerly Honeywell) H1-2 device at $n = 5$ (top left), $n = 6$ (top right), $n = 8$ (bottom left), $n = 9$ (bottom right). Mean HOP and 2σ below the mean are both computed cumulatively using the historical data up to point i along the x-axis.

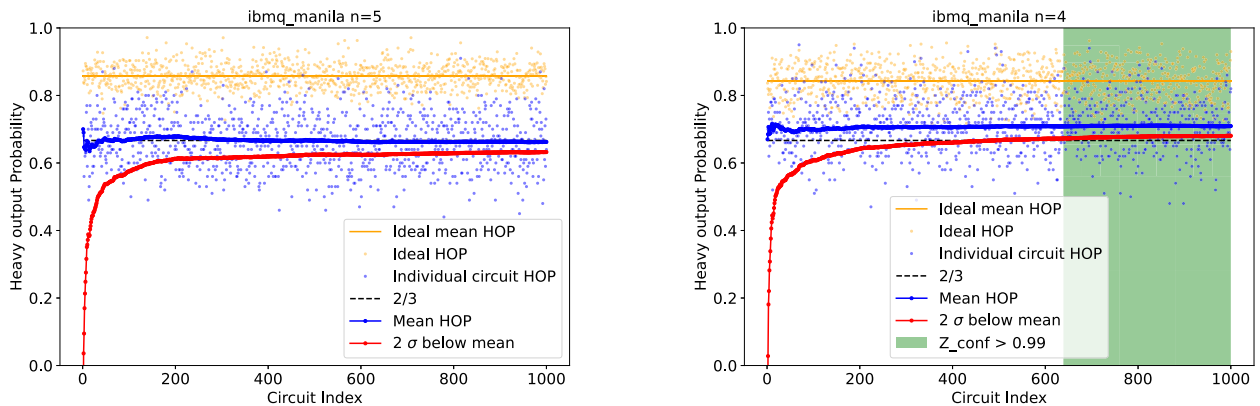


FIGURE 6. HOPs as a function of circuit index for the `ibmq_manila` device using black-box circuit execution for $n = 4$ (bottom) and $n = 5$ (top).

C. BLACK-BOX IONQ

Fig. 7 shows that the 11-qubit IonQ backend passes the QV test at $n = 3$ and fails to pass at $n = 4$. For $n = 3$, execution was stopped once the results passed the z-confidence threshold of 0.99. Because the mean HOP for $n = 4$ was definitively lower than two-thirds, we stopped execution at 500 circuits.

D. BLACK-BOX OQC

Fig. 8 shows the HOP distribution from executing the 1000 QV circuits at $n = 2$ on the OQC Lucy backend. This plot shows that the mean HOP is consistently below two-thirds. In

the compilation of these circuits, the OQC compiler applied optimizations to the circuit(s) that were submitted through Amazon Braket.

E. BLACK-BOX RIGETTI

Fig. 9 shows that using black-box compilation and job submission, the Aspen-11 device fails to pass the QV test at $n = 3$ and $n = 2$. Note that $n = 2$ was tested, unlike the other backends, since the $n = 3$ test failed. We additionally tested the Aspen-M-1 backend for $n = 2$, which also failed to pass the QV protocol.

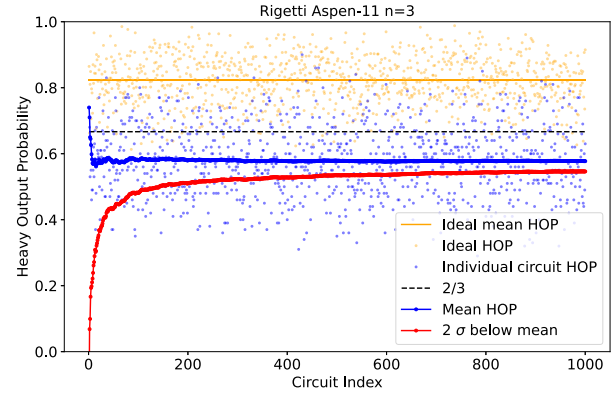
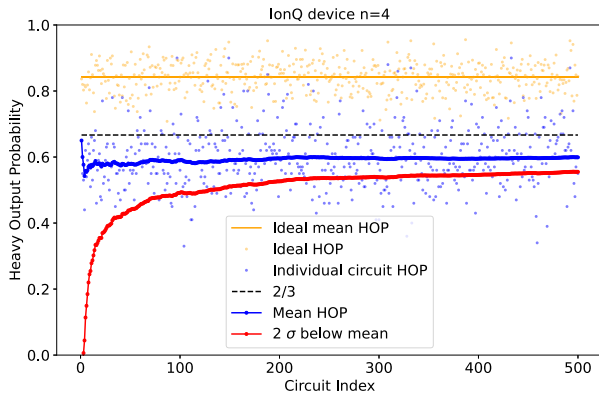
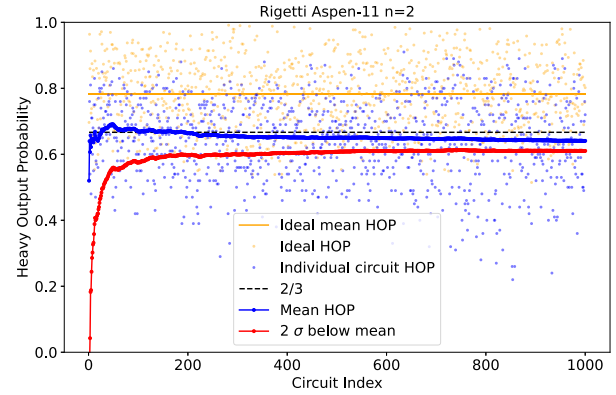
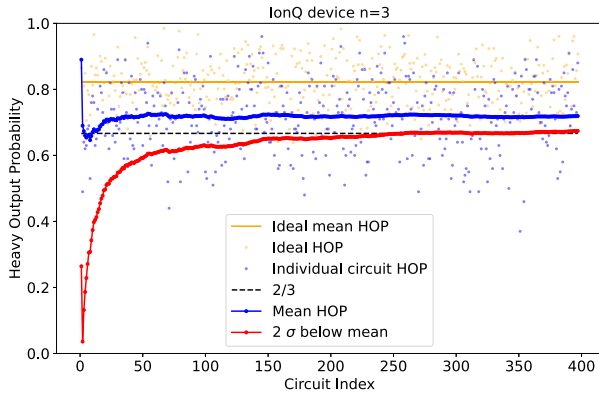


FIGURE 7. IonQ HOP plots for $n = 3$ (top) and $n = 4$ (bottom): IonQ passes the $n=3$ QV protocol at about 380 circuits but is still far from passing at $n=4$ after 500 circuits.

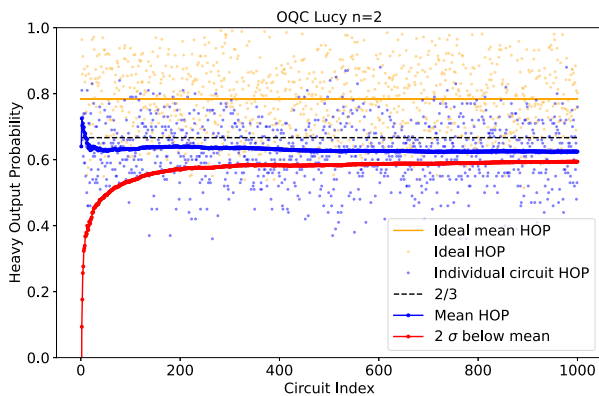


FIGURE 8. OQC Lucy backend HOP plot for $n = 2$ show that the mean HOP is consistently below two-thirds.

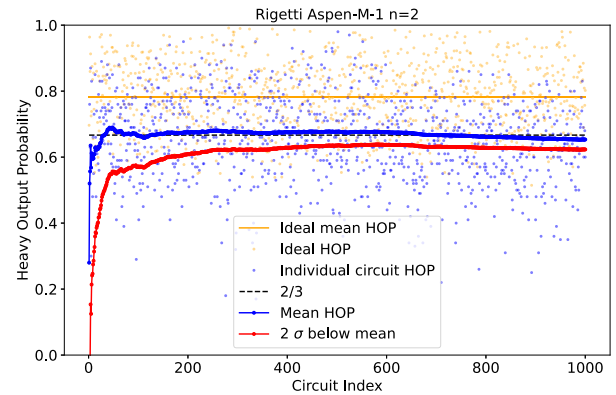


FIGURE 9. Black-box execution of Rigetti Aspen-11 HOP distribution for $n = 2$ (top) and $n = 3$ (middle), and Aspen-M-1 HOP distribution for $n = 2$ (bottom). We see that at $n = 2$, the mean HOP is very close to two-thirds, at times passing two-thirds for a smaller number of circuits, but the 2σ value consistently remains below two-thirds.

F. RIGETTI QUBIT SUBSET ENUMERATIONS

Here, we summarize the results from executing the qubit subset enumeration QV protocol on Aspen-M-1 and Aspen-11 on the QCS platform [35]. Not all qubit subsets of Aspen-11 and Aspen-M-1 were tested. For Aspen-11 at $n = 2$, 12 out of the possible 48 connected qubit subsets were tested, 8 of which passed with a 99% z-confidence level. For Aspen-11 at $n = 3$, 51 out of the possible 72 connected qubit subsets were compiled to and executed, none of which passed with high confidence. For Aspen-M-1 at $n = 3$ 26 out of the possible 184 connected qubit subsets were tested, one of which passed

with a 99% z-confidence level. Fig. 10 shows four specific examples of cumulative HOP distributions where $n = 2$ both passed and failed on Aspen-11, and where $n = 3$ both passed and failed on Aspen-M-1.

G. QISKIT TRANSPILER WITH QUBIT SUBSET ENUMERATION: IBM Q

Table 2 shows the qubit subset enumeration results for each of the IBM Q backends when the QV protocol is applied using the Qiskit transpiler with no modifications to the transpilation procedure; only compilation flag level 3 (which is

TABLE III. Successful QV Measures Across Some IBM Q Backends Compared to the Vendor-Provided $\log_2$ QV, When Using the Custom QV Passmanager for Compilation

ibmq device	manila	bogota	guadalupe	lagos	toronto	hanoi	cairo	montreal	brooklyn
# of qubits	5	5	16	7	27	27	27	27	65
IBM Q $\log_2$ QV	5	5	5	5	5	6	6	7	5
$n = 3$	3/3	3/3	12/20	7/7	27/37	17/37	11/37	34/37	95/95
$n = 4$	2/2	0/2	4/24	4/4	3/16	2/20	0/4	20/30	7/13
$n = 5$	1/1	0/1	4/25	4/4	17/62	6/38	0/17	22/49	31/90
$n = 6$						0/37	0/48	0/36	
$n = 7$								0/3	

Entries in the table have numerator equal to the number of subgraphs that passed the test, and denominator equal to the number of connected subgraphs that we could compile all 1000 QV circuits to in reasonable time.

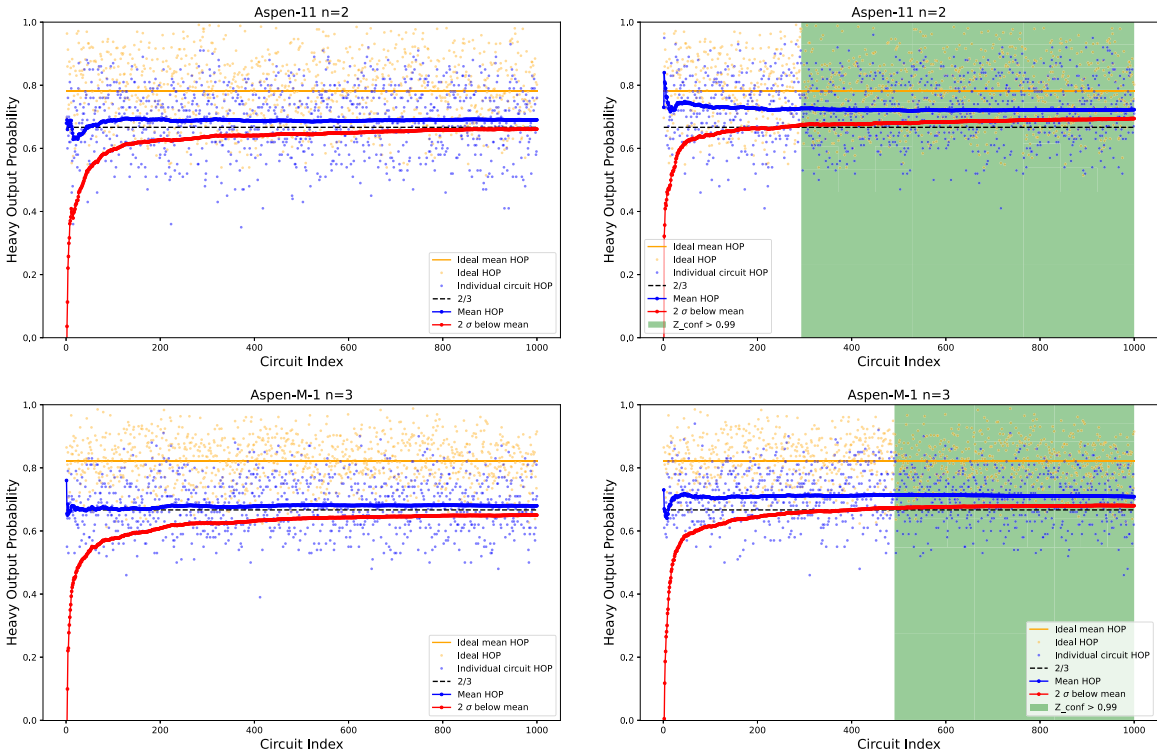


FIGURE 10. Cumulative HOP plots for Rigetti qubit subset enumeration procedure. Top left: $n = 2$ QV circuits run on qubits 14 and 15 of the Aspen-11 device did not pass a mean HOP above two-thirds. Top right: $n = 2$ on qubits 26 and 27 of the Aspen-11 device passed the QV test after roughly 300 circuits. Bottom left: $n = 3$ QV circuits on qubits 146, 131, and 132 of Aspen-M-1 did not pass two-thirds, whereas $n = 3$ QV circuits ran on qubits 112, 113, and 114 passed the test after roughly 500 circuits.

the highest flag supported by this method), the connectivity graph, and the required basis gates are provided as additional transpiler arguments. Because of the size of the backend, only some of the qubit subsets of `ibmq_washington` at $n = 4$ were tested. Running $n = 3, 5$ on `ibmq_washington` would have also required significant additional QPU time.

Fig. 11 shows two HOP plots for two different IBM Q backends at $n = 4$. Importantly, this compilation procedure resulted in the highest QV value found across all tested IBM Q backends was $n = 4$ (QV = 16).

Fig. 12 shows heatmaps of several IBM Q backends in terms of QV protocol success counts across the entire chip. Notably, as with the error rates on the chip, the distribution of higher success rate qubits is not uniform across all qubits.

While this is to be expected, this result shows the importance of backend connectivity and the error rates of particular gates; the QV value for a backend does not necessarily hold for all the qubits on the backend.

Fig. 13 shows the distribution of mean HOP values from the Qiskit transpiled QV circuits across all IBM Q backends and connectivities, organized into three histograms corresponding to $n = 3, 4, 5$. Two observations are noteworthy; first, all three histograms seem to have bimodal characteristics, which could correspond to different processor generations. Second, although no $n = 5$ QV protocol passed z-confidence of 0.99, the histogram shows that some mean HOP values did cross the two-thirds threshold; but the amount over two-thirds was not significant.

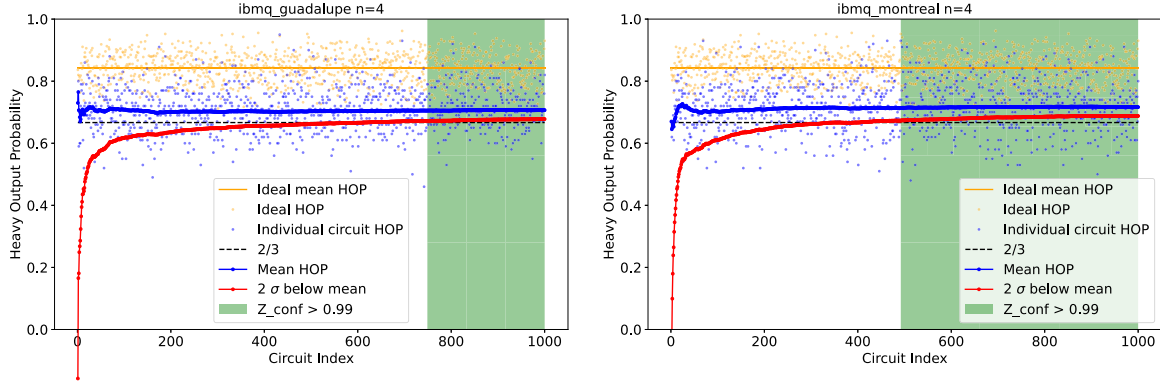


FIGURE 11. IBM Q cumulative HOP on qubits 8, 9, 11, 14 of *ibmq_guadalupe* (left) and qubits 16, 19, 20, 14 of *ibmq_montreal* (right) at $n = 4$. The shaded green regions show where the HOP distribution becomes statistically significant above two-thirds with z-confidence > 0.99 .

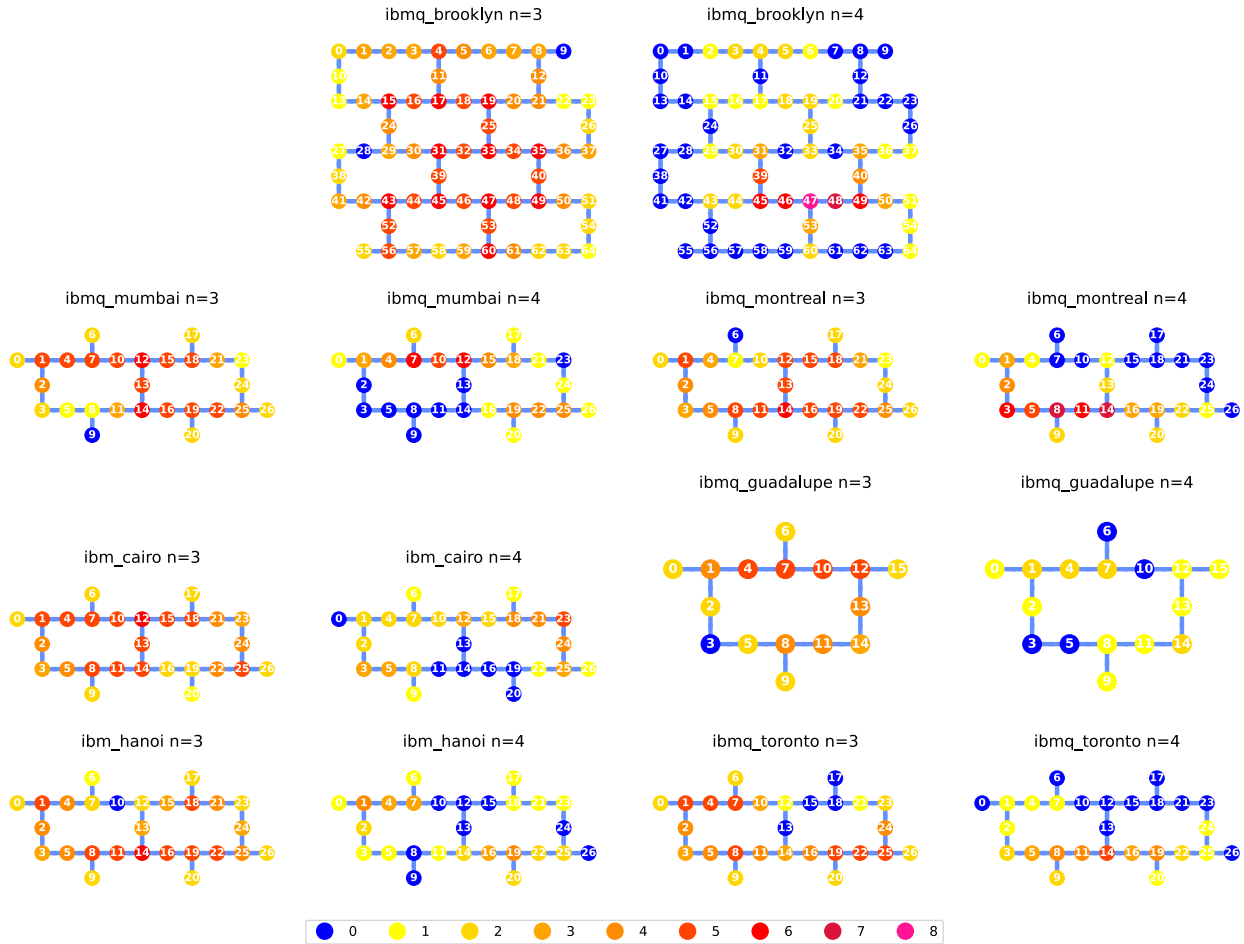


FIGURE 12. Heatmaps showing how many of the successful QV subgraphs each qubit was a member of across several IBM Q backends for $n = 3$ and $n = 4$ sized QV circuits, corresponding to some of the data in Table 2. Due to the volume of circuits executed in order to obtain these data, these results are not fully self consistent because the noise profile of the backend can change over time, and these results were gathered over a time period of up to several weeks.

H. CUSTOM COMPILATION USING QV PASSMANAGER: IBM Q

Table 3 summarizes the QV results on a subset of the IBM Q backends when using the **custom QV passmanager** for circuit compilation. Compilation for *ibmq_mumbai* and *ibmq_auckland* were successful but failed to execute on the

backends due to an internal error relating to the pulse instruction durations. The pulse gate duration and timing needs to be specified very precisely [see Fig. 2, circuit (4) where the QV passmanager specifies Delay gates in order to make the Pulse gates work correctly on the backend]; it appears that the compilation to the *ibmq_mumbai* and *ibmq_auckland*

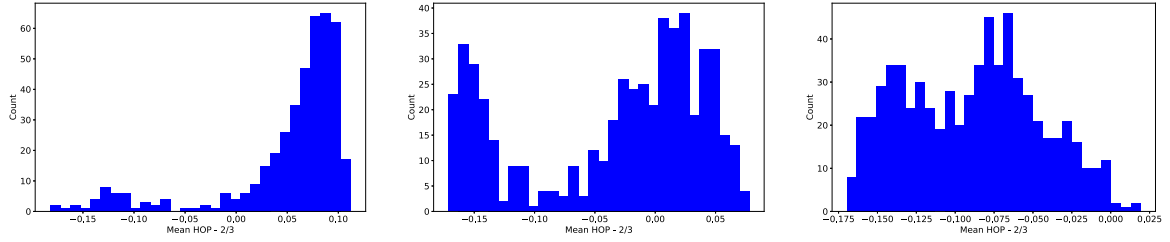


FIGURE 13. Distribution of mean HOP minus two-thirds across all IBM Q backends and connected subgraphs (see the full QV results from this dataset in Table 2) for $n = 3$ (left), $n = 4$ not including `ibm_washington` (middle), $n = 5$ (right) when using the Qiskit transpiler in order to compile the raw circuits onto each backend and qubit subset choice.

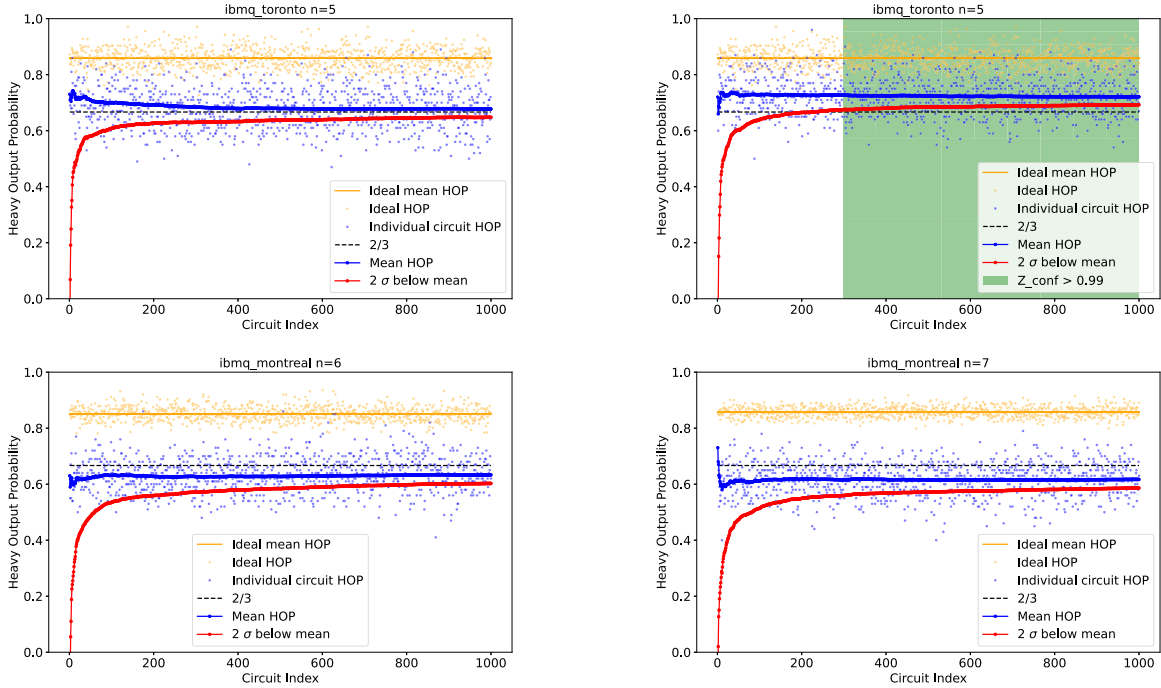


FIGURE 14. IBM Q custom QV passmanager compilation for $n = 5$ on qubits 1, 2, 4, 7, 10 (top left), qubits 3, 5, 8, 11, 14 on `ibmq_toronto` (top right). Then, $n = 6$ with qubits 14, 16, 19, 20, 22, 25 (bottom left) on `ibmq_montreal` and $n = 7$ with qubits 5, 8, 11, 14, 16, 19, 20 (bottom right) on `ibmq_montreal`. These plots show a similar result to the default Qiskit transpiled circuits; the qubit subset choice can change the QV result very significantly.

backends failed because of an error related to the circuit timing. Finally, additional circuits compiled using this custom passmanager were not executed on `ibm_washington` due to the significant QPU time usage it would require.

Importantly, this custom compiler increased the measured QV on many of the backends compared to the default Qiskit transpiled circuits; going from $n = 4$ when using the default Qiskit transpiler, to $n = 5$. However, this custom passmanager did not work on all backends, required heavy computation time, and it did not consistently find the same QV values reported by the vendor, although not all qubit subsets were evaluated on all of the backends we tested. Although it does improve circuit fidelity, the more custom compilation techniques are not feasible for a typical user.

Fig. 14 shows a side-by-side of (different) qubit subset HOP results from `ibmq_toronto` and `ibmq_montreal`, showing how the layout of the qubits can change the QV result.

I. QV OVER TIME

Another important aspect of a benchmark for quantum computers is that the noise profile on the device changes over time, which means that benchmarks on a specific part of the hardware may not be consistent over time. Fig. 15 shows the mean HOP after having run all 1000 QV circuits for $n = 3$ and $n = 4$ on fixed qubit subsets of the `ibm_auckland` device. The mean HOP achieved at each of these time points noticeably varies over the approximately 400 h of the data collection. This shows that at points, this set of qubits fails to pass the $n = 3$ QV test (because the mean HOP is below two-thirds, but at other points, the mean HOP for $n = 4$ goes above two-thirds. This shows that there is significant variation in the QV test over time. Therefore, it is important to show the aggregate behavior of an NISQ device on the QV benchmark, for example, what is shown in Fig. 12.

Although the $n = 3$ and $n = 4$ QV circuits were run with a small time delay in between, we would expect to see high

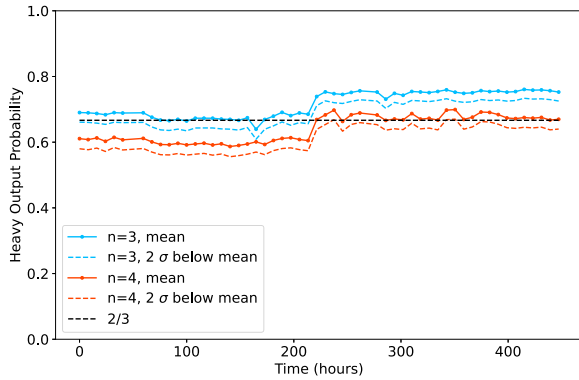


FIGURE 15. Fluctuations of QV over 18 days: For repeated experiments over time on `ibmq_auckland`, we show the measured Mean QV HOP data points (each taken across all 1000 QV circuits of size n). The 2σ below mean line is shown as dashed lines. Importantly, this shows that the measured QV drifts from 4 ($n < 3$) at the start to 8 ($n = 3$) at about 200 h. For a brief time at about 350 h, we have QV 16, as the 2σ below the mean ($n = 4$) line just passes the two-thirds threshold. Experiments for $n = 3$ were run on qubits (10, 12, 13), experiments for $n = 4$ on qubits (10, 12, 13, 14).

correlation between the $n = 3$ and $n = 4$ lines in Fig. 15. We observe this to be the case for the overall trends; however, there are small discrepancies between the two lines; for example, at approximately 180 h, there is a dip in the blue $n = 3$ line but not a corresponding dip in the orange $n = 4$ line.

IV. DISCUSSION

QV is designed to be a benchmark that can compare quantum backends to other quantum backends with different underlying hardware. What we found is that the particular QV protocol used (i.e., how many circuits are run), and how the QV circuit are compiled, massively impacts the measured QV. For end users who will employ the simple compiler methods available in the quantum SDK's of the hardware vendors, the heavy compilation QV results *do not* reflect the expected backend fidelity they would experience when running jobs because they are not using those more advanced compiler options.

The qubit subset enumeration QV results (for IBM Q) reveals a lot more detail than the black-box method. These results give a more detailed analysis of the quantum device's capabilities; not only allowing more accurate comparisons across different backends but also a more detailed picture of which regions of the device give the best results. Fig. 12 shows that the specific qubits used to execute circuits on the backend greatly impact the circuit fidelity. Using the approach of compiling the same circuits to different qubit subsets of a backend allows even greater evaluation of the performance of a backend; applying this methodology to application benchmarks [40], [41] is interesting future work.

Although we were not able to exhaustively evaluate the the more advanced IBM Q compilation features across all backends and qubit subsets, the more advanced

compilation methods clearly increased the measured QV values (see Table 3).

Lower error rates and high connectivity clearly translate to higher quantum volumes. The Quantinuum **H1-2** backend had the lowest overall error rate across all backends we tested, and it also had the highest QV by a significant amount.

We shows that importantly, although not surprisingly, the QV benchmark drifts over time even for fixed qubits. It is known that there is significant time dependence on NISQ noise profiles [22], and therefore, it follows that benchmarks for these devices would also see time dependence. This is notable because it means that NISQ benchmarks also need to take into account the aggregate behavior of the device, not just a subset of qubits.

Other NISQ benchmarks, such as CLOPS for quantifying execution speed, should also continue to be evaluated in order to provide additional context for the performance of these backends. Another future research area is to quantify the correlation between NISQ benchmarks (such as QV) and error metrics such as *aggregate error* [23], [46], crosstalk, and average qubit fidelity. On average, it is clear that error rates, as well as connectivity and compilers, are the primary factors impacting NISQ device performance. However, exactly quantifying the error experienced by a circuit during execution can be difficult because it relies on time-sensitive calibration data, as well as knowing the exact circuit that was executed on the backend. Additionally QV circuit execution can occur over an extended period of time. Therefore, determining the relationship between aggregate error and NISQ benchmarks is an interesting research avenue.

Due to time and usage limitations (in particular the current trapped-ion qubit technology costs more time per sample compared to superconducting qubit technology), we were unable to complete the full QV tests on the Quantinuum H1-2 device. One of the limitations of the QV benchmark in this respect is that it requires many samples and many circuit executions, which is not always feasible on all platforms. One possible modification of the QV protocol would be to make use of data resampling in order to quantify HOP distributions using a smaller number of samples. This has been introduced in the literature [9] and would make the QV protocol more accessible.

Overall, we find that QV gives a good basis for comparing different NISQ backends if the settings for such a comparison are constant. There are many particular details that can impact the the measured QV of an NISQ device. The most significant appears to be the compiler; on the one hand, heavy optimization can yield better circuit fidelity, but on the other hand, a poor choice of qubit layout can significantly hinder the circuit fidelity. Other important settings include how many circuits are used in the test. For instance Fig. 11 shows that the successful measurement of $n = 4$ for `ibmq_guadalupe` would not occur if we executed less than 400 circuits. Therefore, the QV metric is only useful if there is a consistent basis for comparison (i.e., relatively consistent compiler, and consistent experimental settings).

An important takeaway for users of NISQ computers when running near-term applications is that it is prudent to try different qubit subsets on the hardware and to try different compilation strategies (for example, noise adaptive compiler settings versus specifying some qubits to use) in order to get a good sense of what the device's full capabilities are.

Finally, the QV metric is designed specifically for gate model (i.e., universal) quantum computation devices; however, there are restricted quantum computational devices in the NISQ-era including Quantum Annealers [13], [21], [34], [37], [42], [53] and Boson Samplers [1], [7], [25], [57]. Although direct comparisons across these different quantum technologies are not always possible, benchmarks comparing the state of other NISQ-era technology could be useful.

APPENDIX GATE DEFINITIONS

Here, we provide the exact gate definitions of all of the gate operations referenced for compilation to the different backends as well as the native gates on the different backends.

A. 2-QUBIT GATES

We first define the parameterized two-qubit gates used in this article, which are as follows.

- 1) XY is one of the native two-qubit gates for the Rigetti systems, and it is parameterized by an angle θ

$$XY(\theta) = e^{i\theta/2(XX+YY)/2}$$

$$= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta/2) & i\sin(\theta/2) & 0 \\ 0 & i\sin(\theta/2) & \cos(\theta/2) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

For fixed angle $\theta = \pi$, the $XY(\pi)$ gate is also known as *i*SWAP. The *i*SWAP gate is used for illustration purposes to show a Quil compiled circuit in Fig. 2. XY gates are native to the Rigetti hardware but currently not supported in Qiskit, though the *i*SWAP gates in Fig. 2 correspond to actual $XY(\pi)$ gates.

- 2) RXX is an angle parameterized gate used as the only two-qubit gate when compiling the raw QV circuits in Qiskit to be submitted to the 11-qubit IonQ Harmony device through Amazon Braket

$$RXX(\theta)$$

$$= e^{-i\theta/2(XX)}$$

$$= \begin{pmatrix} \cos(\frac{\theta}{2}) & 0 & 0 & -i\sin(\frac{\theta}{2}) \\ 0 & \cos(\frac{\theta}{2}) & -i\sin(\frac{\theta}{2}) & 0 \\ 0 & -i\sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) & 0 \\ -i\sin(\frac{\theta}{2}) & 0 & 0 & \cos(\frac{\theta}{2}) \end{pmatrix}.$$

- 3) CPHASE is one of the native two-qubit gates for the Rigetti systems, and it is parameterized by the angle ϕ

$$CPHASE(\phi) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\phi} \end{pmatrix}.$$

Next, we define the more common and maybe better known unparametrized two-qubit gates, which are as follows.

- 1) CZ is the third two-qubit gate for the Rigetti systems; it is not parameterized

$$CZ = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}.$$

- 2) The CNOT (CX) two-qubit gate is a native two-qubit gate on IBM Q systems and is used at various levels of the compilation procedures; for example, the raw QV circuits are defined entirely in terms of U3 and CX gates

$$CX = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

- 3) The ECR gate is the two-qubit gate used by the OQC Lucy backend. We compile the raw QV circuits to the OQC Lucy native gateset before submitting the circuits to the Amazon Braket system

$$ECR_{q_0, q_1} = \begin{pmatrix} 0 & 0 & 1 & i \\ 0 & 0 & i & 1 \\ 1 & -i & 0 & 0 \\ -i & 1 & 0 & 0 \end{pmatrix}.$$

- 4) ZZ is the two-qubit native gate for the Quantinuum H1-2 device. In the workflow for this article, the QV circuits are not compiled to the Quantinuum gateset on the user side, instead that compilation is done entirely on the server side

$$ZZ = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & i & 0 & 0 \\ 0 & 0 & i & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

B. 1-QUBIT GATES

We again present parametrized gates first, as follows.

- 1) The rz gate is the most commonly used single-qubit gate; it is used by the Quantinuum, IonQ, IBM Q,

OQC, and Rigetti devices

$$\text{rz}(\lambda) = \begin{pmatrix} e^{-i\frac{\lambda}{2}} & 0 \\ 0 & e^{i\frac{\lambda}{2}} \end{pmatrix}.$$

- 2) The ry gate is one of the single-qubit gates we use when compiling the QV circuits to the IonQ gateset

$$\text{ry}(\theta) = \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}.$$

- 3) The rx gate is a single-qubit gate used by both Rigetti and IonQ devices as a native gate

$$\text{rx}(\theta) = \begin{pmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}.$$

- 4) U_1 is one of the native single-qubit gates for the Quantinuum H1-2 device (the other single-qubit gate is rz). As with the ZZ gate, we do not use this gate for compiling and submission to the H1-2 device as the compilation is handled server side

$$U_1(\theta, \phi) = \begin{pmatrix} \cos \frac{\theta}{2} & -ie^{-i\phi} \sin \frac{\theta}{2} \\ -ie^{i\phi} \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}.$$

- 5) The U3 gate is used when the raw QV circuits are generated using Qiskit, as its parameters cover the whole space of single-qubit unitaries. Those raw QV circuits are defined entirely in terms of U3 and CX gates. Subsequent compilations transforms those circuits into other hardware specific gatesets. The compiled circuits returned from the OQC Lucy backend also make use of the U3 gate

$$U_3(\theta, \phi, \lambda) = \begin{pmatrix} \cos \frac{\theta}{2} & -e^{i\lambda} \sin \frac{\theta}{2} \\ e^{i\phi} \sin \frac{\theta}{2} & e^{i(\phi+\lambda)} \cos \frac{\theta}{2} \end{pmatrix}.$$

We conclude with the definitions for fixed single-qubit gates, as follows.

- 1) sx is a native single-qubit gate on IBM Q and OQC devices

$$\text{sx} = \sqrt{X} = \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}.$$

- 2) The Pauli X gate is a native single qubit used on IBM Q and OQC systems

$$x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

- 3) The Hadamard gate is used in description of the compiled QV circuits returned by the OQC device (written in QASM)

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

ACKNOWLEDGMENT

The authors acknowledge the use of IBM Quantum services for this work. The views expressed are those of the authors, and do not reflect the official policy or position of IBM or the IBM Quantum team. This article used resources of the Oak Ridge Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC05-00OR22725. This article also used resources provided by the Darwin testbed at Los Alamos National Laboratory (LANL), which is funded by the Computational Systems and Software Environments subprogram of LANL's Advanced Simulation and Computing program (NNSA/DOE). The authors thank Georg Hahn for discussions on statistical methods to estimate the number of reasonable samples to use for QV benchmarking. Fig. 3 was created in Lucidchart, www.lucidchart.com.

REFERENCES

- [1] S. Aaronson and A. Arkhipov, "The computational complexity of linear optics," in *Proc. 43rd ACM Symp. Theory Comput.*, 2011, pp. 333–342, doi: [10.1145/1993636.1993682](https://doi.org/10.1145/1993636.1993682).
- [2] S. Aaronson and L. Chen, "Complexity-theoretic foundations of quantum supremacy experiments," in *Proc. 32nd Comput. Complexity Conf.*, 2017, pp. 1–67, doi: [10.5555/3135595.3135617](https://doi.org/10.5555/3135595.3135617).
- [3] D. M. Abrams, N. Didier, B. R. Johnson, M. P. da Silva, and C. A. Ryan, "Implementation of XY entangling gates with a single calibrated pulse," *Nature Electron.*, vol. 3, pp. 744–750, 2020, doi: [10.1038/s41928-020-00498-1](https://doi.org/10.1038/s41928-020-00498-1).
- [4] T. Alexander et al., "Qiskit pulse: Programming quantum computers through the cloud with pulses," *Quantum Sci. Technol.*, vol. 5, no. 4, 2020, Art. no. 44006, doi: [10.1088/2058-9565/aba404](https://doi.org/10.1088/2058-9565/aba404).
- [5] MD SAJJID ANIS et al., "Qiskit: An open-source framework for quantum computing," *Qiskit/qiskit*, 2021, doi: [10.5281/zenodo.2573505](https://doi.org/10.5281/zenodo.2573505).
- [6] appleby et al., "rigetti/qvm," 2020, doi: [10.5281/zenodo.3762258](https://doi.org/10.5281/zenodo.3762258).
- [7] J. M. Arrazola et al., "Quantum circuits with many photons on a programmable nanophotonic chip," *Nature*, vol. 591, pp. 54–60, 2021, doi: [10.1038/s41586-021-03202-1](https://doi.org/10.1038/s41586-021-03202-1).
- [8] Atos. Q-Score, 2021. [Online]. Available: <https://github.com/myQLM/qscore>
- [9] C. H. Baldwin, K. Mayer, N. C. Brown, C. Ryan-Anderson, and D. Hayes, "Re-examining the quantum volume test: Ideal distributions, compiler optimizations, confidence intervals, and scalable resource estimations," *Quantum*, vol. 6, p. 707, 2022, doi: [10.22331/q-2022-05-09-707](https://doi.org/10.22331/q-2022-05-09-707).
- [10] M. Benedetti, D. Garcia-Pintos, O. Perdomo, V. Leyton-Ortega, Y. Nam, and A. Perdomo-Ortiz, "A generative modeling approach for benchmarking and training shallow quantum circuits," *NPJ Quantum Inf.*, vol. 5, p. 45, 2019, doi: [10.1038/s41534-019-0157-8](https://doi.org/10.1038/s41534-019-0157-8).
- [11] K. Bharti et al., "Noisy intermediate-scale quantum algorithms," *Rev. Modern Phys.*, vol. 94, 2022, Art. no. 15004, doi: [10.1103/RevModPhys.94.015004](https://doi.org/10.1103/RevModPhys.94.015004).
- [12] R. Blume-Kohout and K. C. Young, "A volumetric framework for quantum computer benchmarks," *Quantum*, vol. 4, p. 362, 2020, doi: [10.22331/q-2020-11-15-362](https://doi.org/10.22331/q-2020-11-15-362).
- [13] S. Boixo et al., "Evidence for quantum annealing with more than one hundred qubits," *Nature Phys.*, vol. 10, pp. 218–224, 2014, doi: [10.1038/nphys2900](https://doi.org/10.1038/nphys2900).
- [14] T. A. Caswell et al., "matplotlib/matplotlib," 2021, doi: [10.5281/zenodo.5194481](https://doi.org/10.5281/zenodo.5194481).
- [15] R. Computing, "Optimizing full-stack throughput and fidelity with Rigetti's Aspen-M generation of quantum processors," Feb. 2022. [Online]. Available: <https://medium.com/rigetti/optimizing-full-stack-throughput-and-fidelity-with-rigetti-s-aspen-m-459ec5b2873f>
- [16] A. Cornelissen, J. Bausch, and A. Gilyén, "Scalable benchmarks for gate-based quantum computers," 2021, doi: [10.48550/arXiv.2104.10698](https://doi.org/10.48550/arXiv.2104.10698).

- [17] A. Cowtan, S. Dilkes, R. Duncan, A. Krajenbrink, W. Simmons, and S. Sivarajah, "On the qubit routing problem," in *Proc. 14th Conf. Theory Quantum Comput., Commun. Cryptography*, 2019, pp. 5:1–5:32, doi: [10.4230/LIPICS.TQC.2019.5](https://doi.org/10.4230/LIPICS.TQC.2019.5).
- [18] IBM ILOG Cplex, "V12.10.0: User's manual for CPLEX," *Int. Business Mach. Corporation*, vol. 46, no. 53, p. 157, 2019.
- [19] A. W. Cross, L. S. Bishop, S. Sheldon, P. D. Nation, and J. M. Gambetta, "Validating quantum computers using randomized model circuits," *Phys. Rev. A*, vol. 100, 2019, Art. no. 32328, doi: [10.1103/PhysRevA.100.032328](https://doi.org/10.1103/PhysRevA.100.032328).
- [20] A. W. Cross, L. S. Bishop, J. A. Smolin, and J. M. Gambetta, "Open quantum assembly language," 2017, doi: [10.48550/arXiv.1707.03429](https://doi.org/10.48550/arXiv.1707.03429).
- [21] A. Das and K. B. Chakrabarti, "Colloquium: Quantum annealing and analog quantum computation," *Rev. Modern Phys.*, vol. 80, pp. 1061–1081, 2008, doi: [10.1103/RevModPhys.80.1061](https://doi.org/10.1103/RevModPhys.80.1061).
- [22] S. Dasgupta and T. S. Humble, "Stability of noisy quantum computing devices," 2021, doi: [10.48550/arXiv.2105.09472](https://doi.org/10.48550/arXiv.2105.09472).
- [23] J. Golden, A. Bärttschi, D. O'Malley, and S. Eidenbenz, "Fair sampling error analysis on NISQ devices," *ACM Trans. Quantum Comput.*, vol. 3, no. 2, 2022, Art. no. 8, doi: [10.1145/3510857](https://doi.org/10.1145/3510857).
- [24] N. Grzesiak et al., "Efficient arbitrary simultaneously entangling gates on a trapped-ion quantum computer," *Nature Commun.*, vol. 11, 2020, Art. no. 2963, doi: [10.1038/s41467-020-16790-9](https://doi.org/10.1038/s41467-020-16790-9).
- [25] C. S. Hamilton, R. Kruse, L. Sansoni, S. Barkhofen, C. Silberhorn, and I. Jex, "Gaussian boson sampling," *Phys. Rev. Lett.*, vol. 119, 2017, Art. no. 170501, doi: [10.1103/PhysRevLett.119.170501](https://doi.org/10.1103/PhysRevLett.119.170501).
- [26] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007, doi: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55).
- [27] IBM, BIPMapping, 2021. [Online]. Available: https://qiskit.org/documentation/locale/ta_IN/stubs/qiskit.transpiler.passes.BIPMapping.html
- [28] IBM, IBM Quantum, 2021. [Online]. Available: <https://quantum-computing.ibm.com/>
- [29] IBM, "Improve Quantum volume via compilation," 2021. [Online]. Available: <https://quantum-computing.ibm.com/services/docs/services/manage/systems/improve-qv/>
- [30] IBM, qiskit-tutorials/qv_tools, Jul. 2021. [Online]. Available: https://github.com/Qiskit/qiskit-tutorials/blob/cac5f9e1eeac695324a71a9d7e9bf268131a1c62b/Tutorials/circuits_advanced/qv_tools.py
- [31] IBM, qiskit.execute_function, 2021. [Online]. Available: <https://qiskit.org/documentation/apidoc/execute.html>
- [32] IBM, "Quantumvolume," 2021. [Online]. Available: <https://qiskit.org/documentation/stubs/qiskit.circuit.library.QuantumVolume.html>
- [33] P. Jurcevic et al., "Demonstration of quantum volume 64 on a superconducting quantum computing system," *Quantum Sci. Technol.*, vol. 6, no. 2, 2021, Art. no. 25020, doi: [10.1088/2058-9565/abe519](https://doi.org/10.1088/2058-9565/abe519).
- [34] T. Kadowaki and H. Nishimori, "Quantum annealing in the transverse ising model," *Phys. Rev. E*, vol. 58, pp. 5355–5363, 1998, doi: [10.1103/PhysRevE.58.5355](https://doi.org/10.1103/PhysRevE.58.5355).
- [35] P. J. Karalekas, N. A. Tezak, E. C. Peterson, C. A. Ryan, M. P. da Silva, and R. S. Smith, "A quantum-classical cloud platform optimized for variational hybrid algorithms," *Quantum Sci. Technol.*, vol. 5, no. 2, 2020, Art. no. 024003, doi: [10.1088/2058-9565/ab7559](https://doi.org/10.1088/2058-9565/ab7559).
- [36] A. Kay, "Quantikz: A Tikz library to typeset quantum circuit diagrams," *Tut. Quantikz Package*, doi: [10.17637/rh.7000520](https://doi.org/10.17637/rh.7000520).
- [37] A. D. King et al., "Scaling advantage over path-integral Monte Carlo in quantum simulation of geometrically frustrated magnets," *Nature Commun.*, vol. 12, 2021, Art. no. 1113, doi: [10.1038/s41467-021-20901-5](https://doi.org/10.1038/s41467-021-20901-5).
- [38] R. LaRose, A. Mari, V. Russo, D. Strano, and W. J. Zeng, "Error mitigation increases the effective quantum volume of quantum computers," 2022, doi: [10.48550/arXiv.2203.05489](https://doi.org/10.48550/arXiv.2203.05489).
- [39] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-Era quantum devices," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2019, pp. 1001–1014, doi: [10.1145/3297858.3304023](https://doi.org/10.1145/3297858.3304023).
- [40] T. Lubinski et al., "Application-oriented performance benchmarks for quantum computing," 2021, doi: [10.48550/arXiv.2110.03137](https://doi.org/10.48550/arXiv.2110.03137).
- [41] D. Mills, S. Sivarajah, T. L. Scholten, and R. Duncan, "Application-motivated," *Holistic Benchmarking a Full Quantum Comput. Stack. Quantum*, vol. 5, p. 415, 2021, doi: [10.22331/q-2021-03-22-415](https://doi.org/10.22331/q-2021-03-22-415).
- [42] S. Morita and H. Nishimori, "Mathematical foundation of quantum annealing," *J. Math. Phys.*, vol. 49, 2008, Art. no. 125210, doi: [10.1063/1.2995837](https://doi.org/10.1063/1.2995837).
- [43] G. Nannicini, L. S. Bishop, O. Gunluk, and P. Jurcevic, "Optimal qubit assignment and routing via integer programming," *ACM Trans. Quantum Comput.*, to be published, doi: [10.1145/3544563](https://doi.org/10.1145/3544563).
- [44] E. Pelofske, A. Bärttschi, and S. Eidenbenz, "NISQ Benchmarking: Quantum volume in practice," 2022. [Online]. Available: <https://github.com/lanl/Quantum-Volume-in-Practice>
- [45] E. Pelofske, A. Bärttschi, and S. Eidenbenz, "Quantum volume in practice: What users can expect from NISQ devices dataset," 2022. [Online]. Available: <https://zenodo.org/record/6360668#.YjtlmsZMHfY>
- [46] E. Pelofske, J. Golden, A. Bärttschi, D. O'Malley, and S. Eidenbenz, "Sampling on NISQ devices: 'Who's the fairest one of all?'," in *Proc. IEEE Int. Conf. Quantum Comput. Eng.*, 2021, pp. 207–217, doi: [10.1109/QCE52317.2021.00038](https://doi.org/10.1109/QCE52317.2021.00038).
- [47] J. M. Pino et al., "Demonstration of the trapped-ion quantum CCD computer architecture," *Nature*, vol. 592, pp. 209–213, 2021, doi: [10.1038/s41586-021-03318-4](https://doi.org/10.1038/s41586-021-03318-4).
- [48] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018, doi: [10.22331/q-2018-08-06-79](https://doi.org/10.22331/q-2018-08-06-79).
- [49] T. Proctor, K. Rudinger, K. Young, E. Nielsen, and R. Blume-Kohout, "Measuring the capabilities of quantum computers," *Nature Phys.*, vol. 18, pp. 75–79, 2022, doi: [10.1038/s41567-021-01409-7](https://doi.org/10.1038/s41567-021-01409-7).
- [50] J. Rahamim et al., "Double-sided coaxial circuit QED with out-of-plane wiring," *Appl. Phys. Lett.*, vol. 110, no. 22, 2017, Art. no. 222602, doi: [10.1063/1.4984299](https://doi.org/10.1063/1.4984299).
- [51] K. Rolston-Duce, "Demonstrating benefits of quantum upgradable design strategy: System model H1-2 first to prove 2,048 quantum volume," Dec. 2021. [Online]. Available: <https://www.quantinuum.com/pressrelease/demonstrating-benefits-of-quantum-upgradable-design-strategy-system-model-h1-2-first-to-prove-2-048-quantum-volume>
- [52] K. Rolston-Duce, "Quantinuum announces quantum volume 4096 Achievement," Apr. 2022. [Online]. Available: <https://www.quantinuum.com/pressrelease/quantinuum-announces-quantum-volume-4096-achievement>
- [53] G. E. Santoro, R. Martoňák, E. Tosatti, and R. Car, "Theory of quantum annealing of an Ising spin glass," *Science*, vol. 295, no. 5564, pp. 2427–2430, 2002, doi: [10.1126/science.1068774](https://doi.org/10.1126/science.1068774).
- [54] R. S. Smith, "Quil: A portable quantum instruction language," 2020, doi: [10.5281/zenodo.3677541](https://doi.org/10.5281/zenodo.3677541).
- [55] R. S. Smith, M. J. Curtis, and W. J. Zeng, "A practical quantum instruction set architecture," 2016, doi: [10.48550/arXiv.1608.03355](https://doi.org/10.48550/arXiv.1608.03355).
- [56] R. S. Smith, E. C. Peterson, M. G. Skilbeck, and E. J. Davis, "An open-source, industrial-strength optimizing compiler for quantum programs," *Quantum Sci. Technol.*, vol. 5, no. 4, 2020, Art. no. 44001, doi: [10.1088/2058-9565/ab9acb](https://doi.org/10.1088/2058-9565/ab9acb).
- [57] N. Spagnolo et al., "Experimental validation of photonic boson sampling," *Nature Photon.*, vol. 8, pp. 615–620, 2014, doi: [10.1038/nphoton.2014.135](https://doi.org/10.1038/nphoton.2014.135).
- [58] N. Sundaresan, I. Lauer, E. Pritchett, E. Magesan, P. Jurcevic, and J. M. Gambetta, "Reducing unitary and spectator errors in cross resonance with optimized rotary echoes," *PRX Quantum*, vol. 1, 2020, Art. no. 020318, doi: [10.1103/PRXQuantum.1.020318](https://doi.org/10.1103/PRXQuantum.1.020318).
- [59] A. Wack et al., "Quality, speed, and scale: Three key attributes to measure the performance of near-term quantum computers," 2021, doi: [10.48550/arXiv.2110.14108](https://doi.org/10.48550/arXiv.2110.14108).
- [60] K. Wright et al., "Benchmarking an 11-qubit quantum computer," *Nature Commun.*, vol. 10, 2019, Art. no. 5464, doi: [10.1038/s41467-019-13534-2](https://doi.org/10.1038/s41467-019-13534-2).
- [61] A. Zulehner and R. Wille, "Compiling SU(4) quantum circuits to IBM QX architectures," in *Proc. 24th Asia South Pacific Des. Automat. Conf.*, 2019, pp. 185–190, doi: [10.1145/3287624.3287704](https://doi.org/10.1145/3287624.3287704).